



MIDDLESEX Community College

Tools and Technologies for Tech Writers 2025

Git Cheatsheet

Notices

This document was prepared as a handout for the Middlesex Community College Tools and Technologies for Technical Writers class, Winter semester 2025.

Prepared by Zoë Lawson, course instructor.

Contents

Introduction.....	4
Formatting conventions.....	4
 Common Git Workflow.....	 6
Recommended Workflow for Class.....	8
Practice, practice, practice.....	11
Gotchas.....	11
 GitHub Desktop.....	 13
Clone Repository.....	14
Change Branch.....	15
A few notes on GitHub Desktop.....	15
Pull changes from remote repository.....	16
Commit changes.....	17
Push changes to remote repository.....	18
Sync a forked repository.....	19
Start a Pull Request.....	20
 GitHub Web Interface.....	 21
Fork a GitHub repository.....	21
Make a pull request.....	23
Sync a forked repository (fetch upstream).....	24
 Command Line.....	 26
Common Git Commands.....	26
Uncommon Git Commands.....	28
Git Status Results.....	30
 Troubleshooting.....	 32
Unable to unlink file.....	32

Introduction

This document is meant to be a companion to the Week 1 handout, `using_git.pdf`.

This document provides a bit more information about the Git commands you will be using commonly in this course.

Remember, you can always go to the internet to search for an answer, or go directly to the available documentation:

- GitHub Help: <https://help.github.com/en/github>
- GitHub Desktop Help: <https://help.github.com/en/desktop>
- Git Command Line Help: <https://git-scm.com/doc>
- *Version Control with Git* from O'Reilly publishing. If you want to deep dive into how Git implements version control, this is the book that helped me the most.

I search the internet for how to do things all the time. There is a lot of fantastic information out there. However, think before you use it. While I have used the information to solve many problems, I have also broken things really spectacularly as well. Code wisely.

In this section:

- [Formatting conventions](#) on page 4
When describing code examples, this document uses the following conventions.

Formatting conventions

When describing code examples, this document uses the following conventions.

Format	Convention
<code>monospace text</code>	A command to run
<code><variable name></code>	Something that needs to be replaced with an actual value. For example, <code><your name></code> should be replaced with whatever your name is.
<code>code to run</code>	An example command that you should enter at a command prompt (or GitBash window) This is also used for code samples, for example, HTML or Markdown examples.
<code>messages and stuff</code>	An example of what a command prompt returns, such as status or error messages

This is an example

Content formatted with this background is an example.

This document has been updated from a previous class. Some images may show out of date branch names.

Common Git Workflow

Git is both a source control tool and a collaboration tool. You need to develop good habits to make it easier for you to work with Git.

Whatever source control tool you use, you need to get comfortable with it. You're going to have to work with it, so it's much better to make peace with it than be antagonistic, or scared of it.

A good habit with any source control (and most everything) is to keep it up to date. In the first place, you keep your skills up using the tool. In the second place, you don't get so far out of sync that updating becomes a big deal. (Where big deal means lost work and tear filled phone calls at 9 PM the day before release.)

1. When you start working, make sure you have the latest and greatest files.

- In this course, that means syncing your repo (*YourGitHubAccount/mcc_tools_tech*) with the source/upstream repo (*ZoeLawson/mcc_tools_tech*). Make sure that both your cloud repo and your local repo are updated.
- In other places, that's usually pulling from main (f.k.a master) or whichever branch has been defined by your development environment. For example, I have worked with a branch called *ProductName_VersionNumber* and with another group, the main branch is called *Development*.

You want to have the latest and greatest changes to make sure you're not missing anything. Someone else might have reworked a section you're also working on. There may be some template or infrastructure changes, such as shared common files.

If you don't sync often, Git might think that your old versions of files are new revisions. A large amount of work can be confused or lost. (Ask me how I know this.)

2. After doing "enough" work, check your changes in.

"Enough" is based on what you're doing. It could be that you fixed the one typo you had to fix. It could be the end of the day and you want to have all your changes stored just in case something terrible happens to your laptop. You might need to switch projects, so better to save everything before going over to the other thing you need to do.

- In this course, you (add and) commit your changes to your local repository.
- In other places, it's generally the same thing, you commit or check in your changes locally.

3. At some point, you have "enough" changes (or a reason) to get your changes to the official repository.

- In this course, that's a two step process. First you push your changes to your repository in GitHub. This means your files are backed up in the cloud. It also means they're staged for going to the 'official' repository. Then you make a pull request to get your changes up to *ZoeLawson/mcc_tools_tech*.
- In other places, you generally need to do the same thing. Push your changes to your area of the hosted repository/server, then make a pull request to get your changes into the official repository.

Be aware that before you merge up, you need to grab the latest and greatest again. You never know when someone else made changes. Also, Git generally doesn't let you push if you don't have the latest and greatest.

So, here is my general habit. I'm using command-line commands because that's the most precise.

1. Get coffee, login, check email, and get the latest greatest from the main repository.

```
git status
```

I always do a status to confirm what branch I am on. I want to make sure I'm working where I think I'm working. In GitHub Desktop, I'd look to see the repo and the branch.

```
git pull origin main
```

Now I should have all the files from people who were working while I was sleeping.

2. Work. Make changes, write stuff, input review comments, etc.
3. When I've done "enough", I make sure my files are saved to my local repository.

```
git add .
```

I may do this several times, I may do this once. Depends on what's going on.

4. Once I've added enough things, I commit the files.

```
git commit -m "Always add a descriptive comment"
```

I may do the add and commit in stages. All the files I've added get the same commit message, so I might add three files with one commit message ("comments from Bob") and then add seven more files and commit them with a different message ("Update input button to submit per ABC-12345")

5. When I've made enough changes, I get my files up to my repo up on the server.

```
git push origin My_Branch
```

Now my files are backed up on the server.

Often there's automation set up, and I can now test build my changes. Then I can review my PDF or Help or whatever. If I like what see, I move on to the next step. Otherwise I go back to revising, committing, and pushing.

6. I'm happy with my changes, and relatively sure I'm not going to break anything. Time for a pull request.

Making a pull request generally happens in whatever web tool I'm using. There isn't really a command-line way to make one. We go up into GitHub.com and click **Create pull request**. At a previous company I went to BitBucket, at a different company, I went into GitLab.

In this section:

- [Recommended Workflow for Class](#) on page 8
Every time you sit down to do class work, you should make sure your Git repository is up to date.
- [Practice, practice, practice](#) on page 11
I cannot stress enough how much easier using Git becomes with using it frequently.
- [Gotchas](#) on page 11
This is in no way a comprehensive list, but here are some Git things that really confused me and took time to comprehend.

Recommended Workflow for Class

Every time you sit down to do class work, you should make sure your Git repository is up to date.

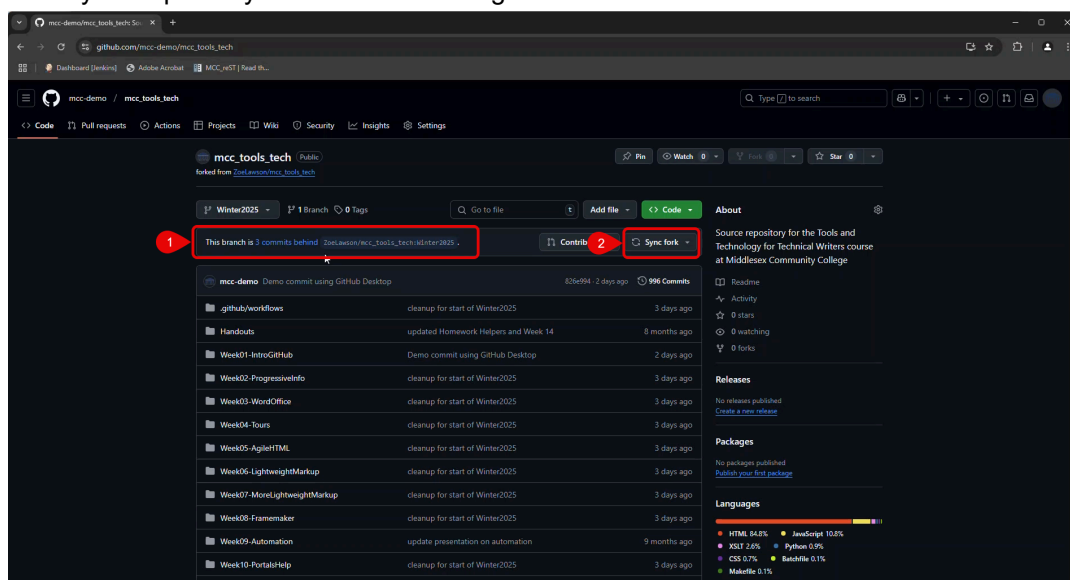
You do not need to worry about following the workflow the first week. In the first place, I'm trying not to overwhelm you too much, so I'm not going to explain how to do this until the second week. In the second place, since you just cloned the repository, there aren't any new files you need just yet.

Starting the second week, you will want to follow this (or a similar) workflow.

Tip: Double check that you don't have any PDFs open. Potentially, close Adobe Acrobat as well.

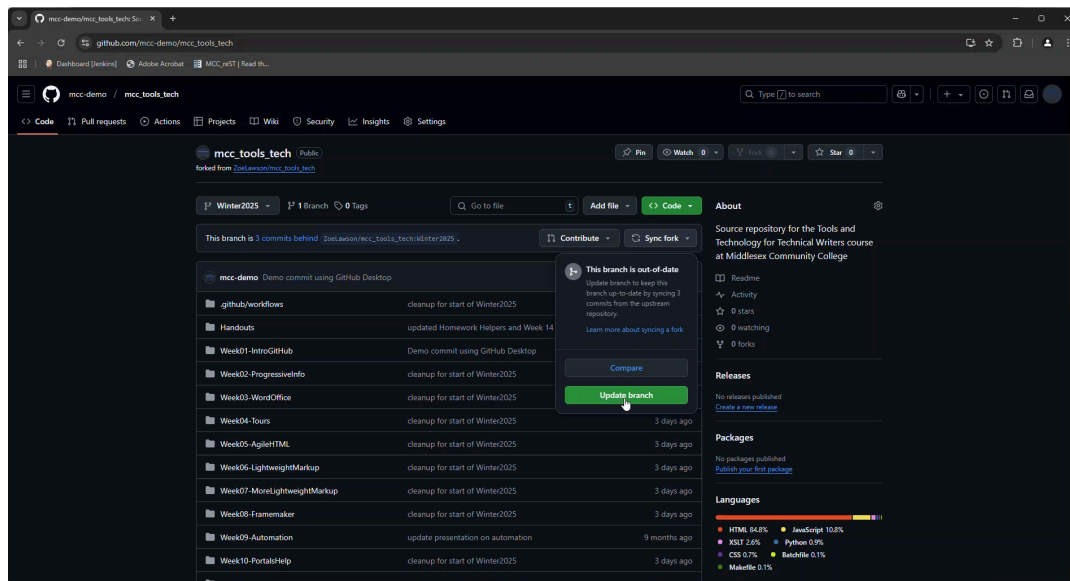
Adobe Acrobat likes to put locks on files it has open. If you have a class PDF open in when you update your files, and there's a change to that PDF, Git has issues.

1. Go to your repository at GitHub.com using a web browser.



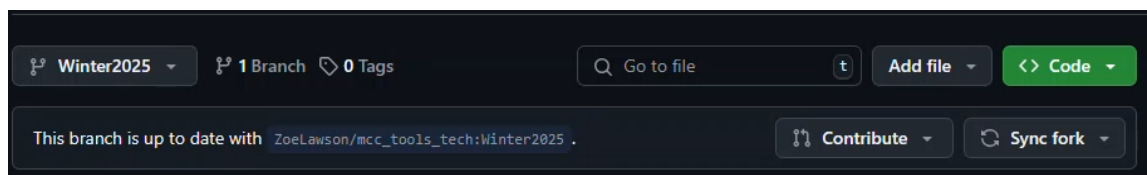
Notice that this fork is 3 commits behind. That means there are changes in the class repository you don't have.

2. Click **Sync Fork**.

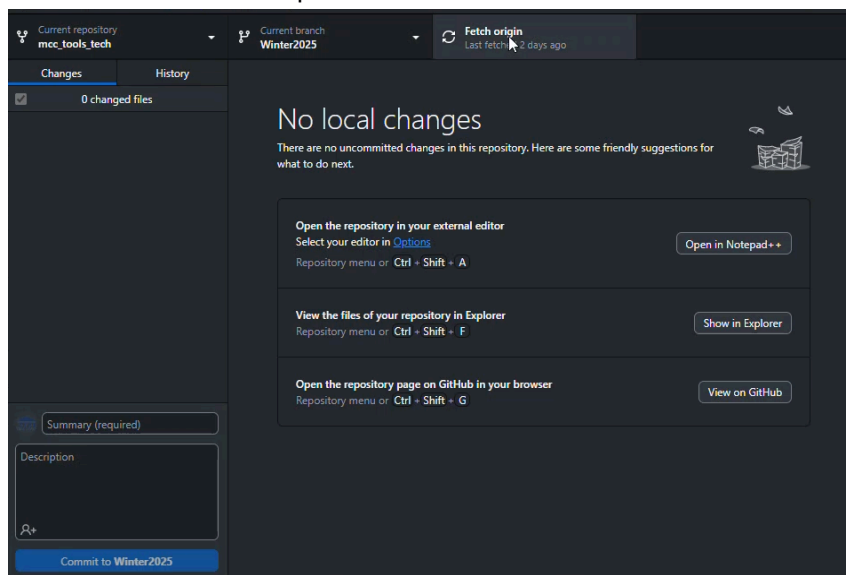


3. Click **Update Branch**.

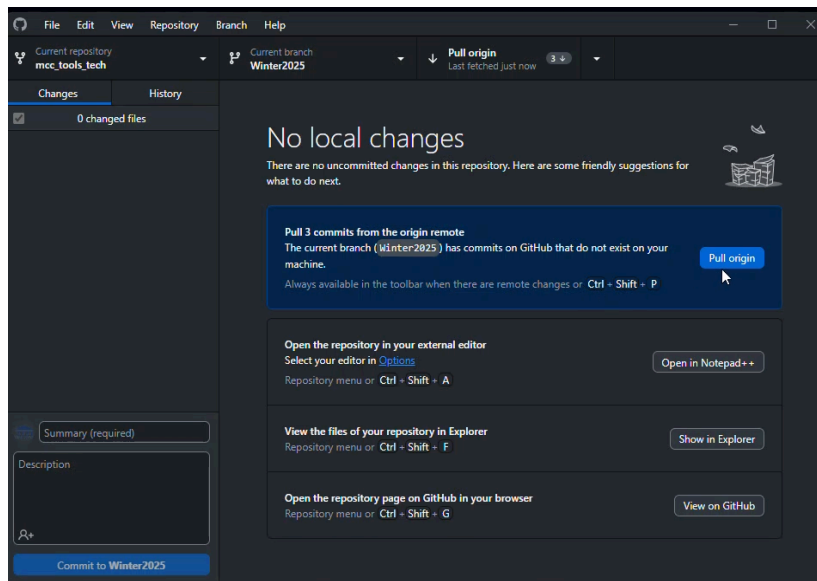
When the update completes, you can see the status change.



4. Switch to GitHub Desktop.

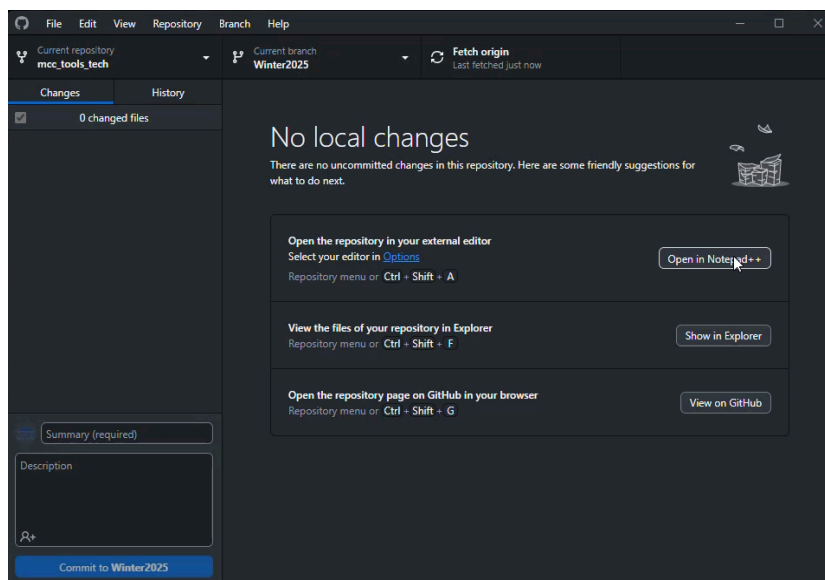


5. Click **Fetch origin**.



6. Click **Pull origin**.

GitHub Desktop downloads the updates from your fork.



Ta-da! You now have the latest and greatest files on your system.

Believe it or not, the image in step 4 is different from step 6.

Have I mentioned that Git is confusing?

However, you are now ready to work on your homework.

Practice, practice, practice

I cannot stress enough how much easier using Git becomes with using it frequently.

I no longer use Git at my work place. I have to refresh my memory every year to teach this class.

Even when I used it daily, I still had to look up certain tasks. When things go sideways, I definitely have to look up how to troubleshoot and fix things.

Ask

There is ZERO shame in asking how to do a thing.

You are learning. You are not going to get everything instantly.

At a former position, I was the person who trained the rest of my team on how to use Git. The first three months my coworkers asked me daily how to do things. Some people asked a question once a week, other folks asked me every single day. I never minded. I still don't.

When I'm doing something important, I double (and triple) check myself.

Make a safety net

Backups are your friends.

Backups generally take seconds and can save you hours and days of work.

- Make a copy of files or folders and store it separately from where you are working.
- Make a zip file (Compressed folder) of the files or folders and store it separately from where you are working.

Tip: Do not store your backups as children of what you are working on. For example, if you are working in MyWork, don't save your backups in MyWork/Backups.

Minimally, put the files in a completely separate folder. Best practice is to use a completely different drive. That drive can be on the cloud. Whatever you choose, be mindful of security policies and practices.

Some companies are blocking the use of thumb drives on their systems. If your content is on the cloud, make sure the security settings of your cloud storage are what you want.

Recovery is possible

If you follow best practices, you can usually recover your work, or at least revert to a save point.

Worst case scenario, you can delete your entire repository locally and re-clone it. If you completely confuse your system, sometimes that's the best answer.

Gotchas

This is in no way a comprehensive list, but here are some Git things that really confused me and took time to comprehend.

You don't need to read this initially. However, these are things that may make sense later.

Commits are everything

I generally look at files and folder and think in files and folders. I also like to look at time stamps. In source control, I think of the history of the file as version numbers of files. Most Git GUIs do the same thing.

In Git, the commit is the system of record.

EVERYTHING is tied to the commit.

Using Git command line, you cannot easily get version 7 of a file. You could revert back to the commit that made version 7...but then you lose all the other work everywhere else. You might be able to *cherry pick* the commit that made version 7 of that file, but it would also get all the other files in that commit.

When you're dealing with code, if you keep all your commits discrete to specific tasks, this works pretty well. Working with documentation where I'm often drafting features x, y, and z, incorporating feedback from a review or two, and fixing a whole bunch of old content and sometimes commit 5 documents at time, this doesn't work as well.

One branch at a time

This won't be as important for this course, but is very important when working with many different branches.

When you clone a repository, you can only see one branch at a time.

When you checkout (switch) a different branch, the root directory stays the same but all the files change.

Without having two clones of a repository on your system, with each set to different branches, you can't have the same file from different branches visible at the same time.

This can be very important when working with the same file in different branches.

GitHub Desktop

There are many different applications you can use to interact with Git besides the command line. One option is the GitHub Desktop, which we are using in this course.

You can download GitHub Desktop from <https://desktop.github.com/>.

The help for GitHub Desktop is available from <https://help.github.com/en/desktop>.

I have used a couple different Git GUIs over the years. Also, several major authoring tools, such as MadCap Flare and OxygenXML, have their own Git interfaces so you can commit files from within the authoring tool. Learn to use the tool your company recommends.

In this section:

- [Clone Repository](#) on page 14
This is generally a one-time task to make a copy of the remote repository in GitHub on your local system.
- [Change Branch](#) on page 15
By default, Git starts in the default branch after you clone a repository. That should be Winter2025, but it may not be.
- [A few notes on GitHub Desktop](#) on page 15
A few tips on what the GitHub Desktop is trying to show you. This screen shows if you have no local changes.
- [Pull changes from remote repository](#) on page 16
If there are changes to the remote repository, you need to pull them into your local repository to make sure you have the latest and greatest changes.
- [Commit changes](#) on page 17
GitHub desktop automatically determines when you've added or changed files in your local copy of the repository. You can easily Commit the changes to your local repository.
- [Push changes to remote repository](#) on page 18
After you've committed changes to your local repository, you need to push them to the remote repository.
- [Sync a forked repository](#) on page 19
You forked the ZoeLawson/mcc_tools_tech repository. When I make changes to the original ZoeLawson/mcc_tools_tech repository, you need to sync the files to get the changes into your forked version of the repository.
- [Start a Pull Request](#) on page 20
To make a Pull Request, you have to use GitHub.com. GitHub Desktop provides a shortcut.

Clone Repository

This is generally a one-time task to make a copy of the remote repository in GitHub on your local system.

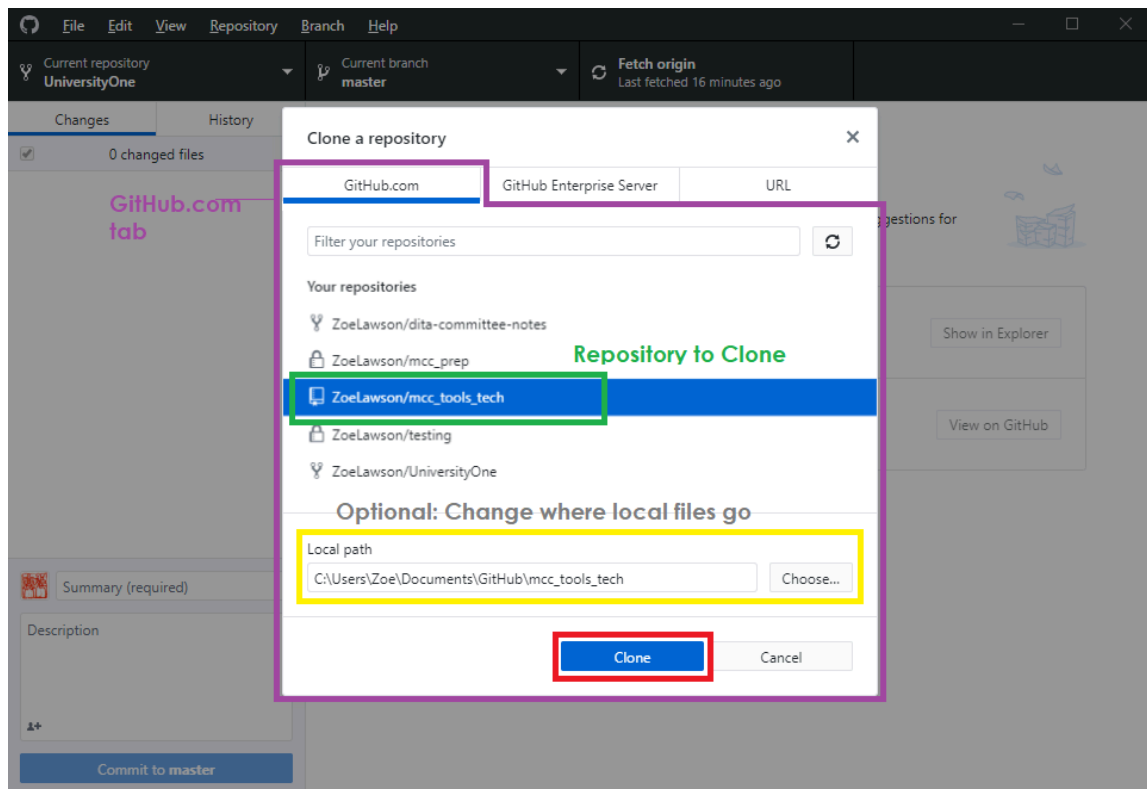
You must have the repository in your GitHub account before you begin. This can be a new repository you create for your own project, or a fork of an existing project.

In this class, you will be working with a forked repository. See [Fork a GitHub repository](#) on page 21.

1. Select **File > Clone repository**.
2. Select the repository you want to clone.

For this class, you want to select your fork of the `mcc_tools_tech` repository on the GitHub.com tab.

This example is from my account, so the name is ZoeLawson. Your version of GitHub Desktop should show your GitHub account name.



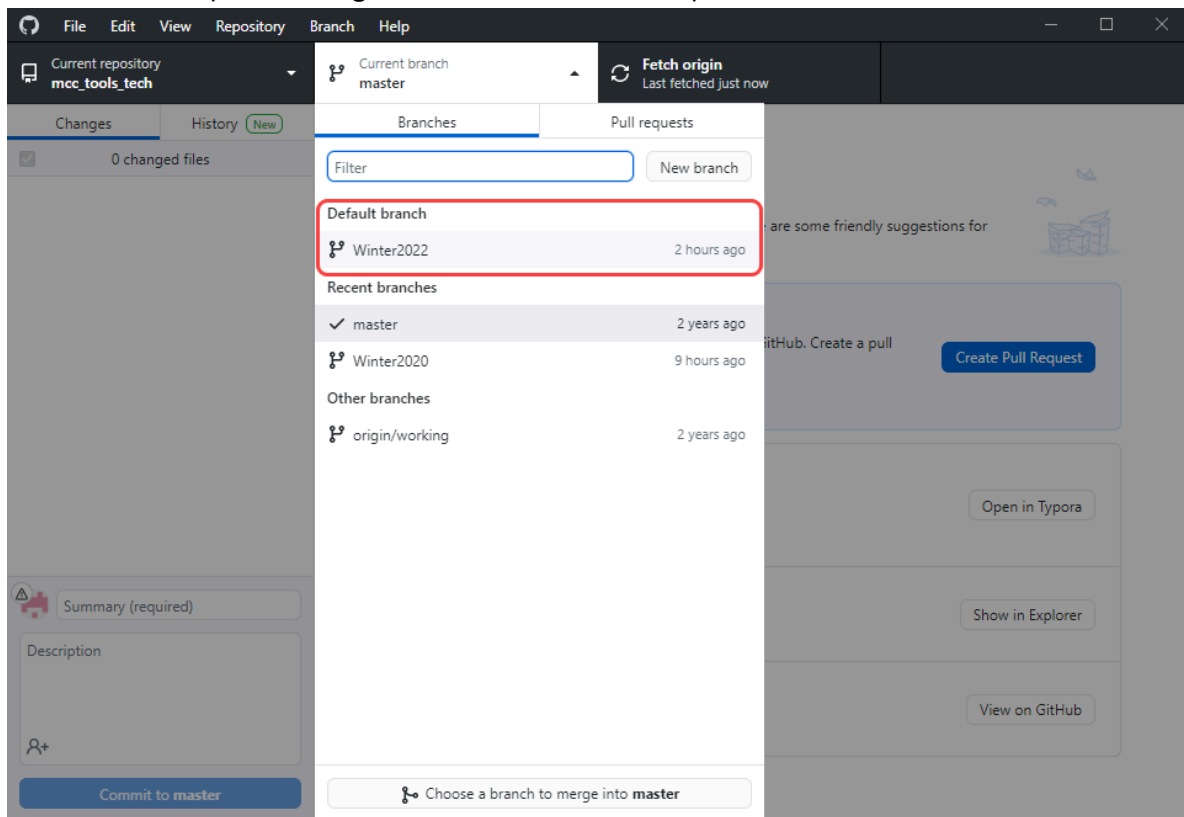
3. Change where the files are stored locally by changing the **Local path**.
4. Click **Clone**.

Change Branch

By default, Git starts in the default branch after you clone a repository. That should be Winter2025, but it may not be.

In this class, you want to work in Winter2025.

In GitHub Desktop, select **origin/Winter2025** from the drop-down under **Current branch**.



A few notes on GitHub Desktop

A few tips on what the GitHub Desktop is trying to show you. This screen shows if you have no local changes.

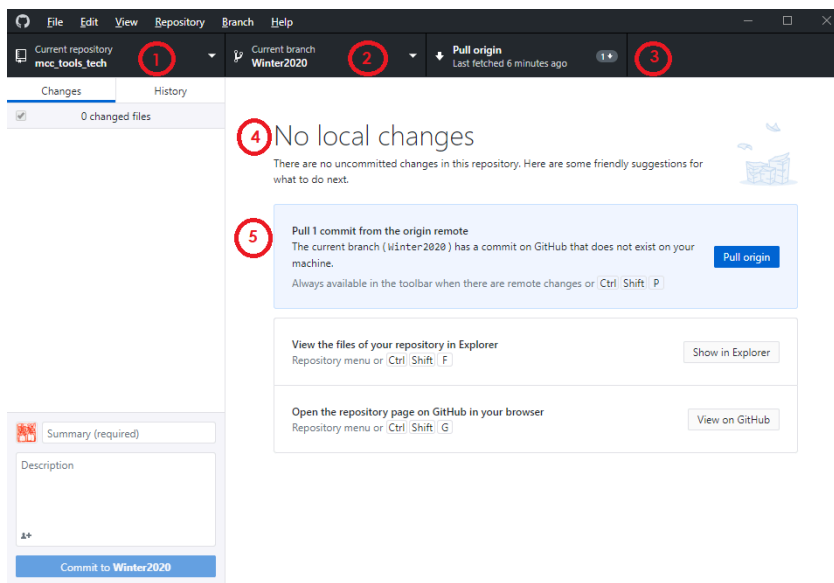


Figure 1: GitHub Desktop recommending a Pull

1. The name of the repository you are currently looking at. For this class, you only need to point to one repository, `mcc_tools_tech`. If you play around on GitHub, you may have many more.
2. The name of the branch you are currently working with. This should be `Winter2025` for this class.
3. The last button changes as the status of the remote repository changes.
 - You can **Fetch** changes from the remote repository (called "origin"). This is how GitHub Desktop figures out if there are changes available in remote repository.
 - You can **Pull** updates from the remote repository (called "origin") down to your local system. This is how you get updates from the remote repository. (This is different from syncing a forked repository. See [Sync a forked repository](#) on page 19.)
4. Gives you the status of local changes on your system.
5. Suggested command based on the current file situation. This example shows that there are changes in the remote repository you should get locally. Therefore, it recommends you **Pull origin**. If you had local changes, it suggests to **Push** files to origin.

Pull changes from remote repository

If there are changes to the remote repository, you need to pull them into your local repository to make sure you have the latest and greatest changes.

This is different than syncing a forked repository. See [Sync a forked repository](#) on page 19.

GitHub Desktop offers three different methods to pull changes into your local repository. At least one of these options is always available. If you can't find the blue **Pull origin** button, you can use the **Repository** menu command.

- Click the blue **Pull origin** button. (5 in [Figure 1: GitHub Desktop recommending a Pull](#) on page 16.)

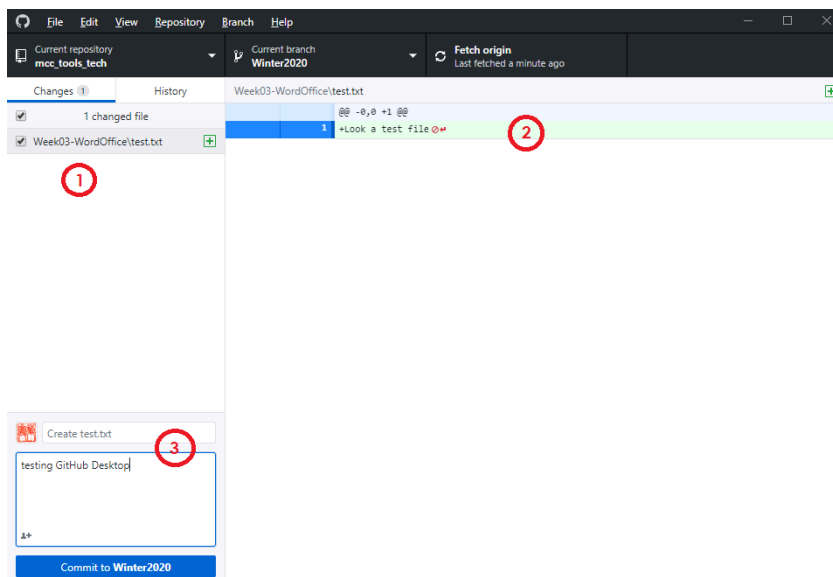
- Click the black **Pull origin** button. (3 in [Figure 1: GitHub Desktop recommending a Pull](#) on page 16.)
- Select the menu **Repository > Pull**.

GitHub Desktop updates your local files with the latest and greatest files from the remote repository (origin).

If there are no changes to pull, the buttons may say **Fetch origin**. After you click **Fetch origin**, if there are changes, the buttons should change to **Pull origin**.

Commit changes

GitHub desktop automatically determines when you've added or changed files in your local copy of the repository. You can easily Commit the changes to your local repository.



1. The Changes panel lists all the files that have changed. There will be different indicators for new , changed , or deleted  files.
2. If the file is a type of file GitHub Desktop knows how to handle, such as a text file, it will show the contents of the file and mark the changes.
3. Where you can provide a summary of changes (in this example Create test.txt) and optionally, provide more details (in this example, testing GitHub Desktop).

Use GitHub Desktop to commit changes to your local repository.

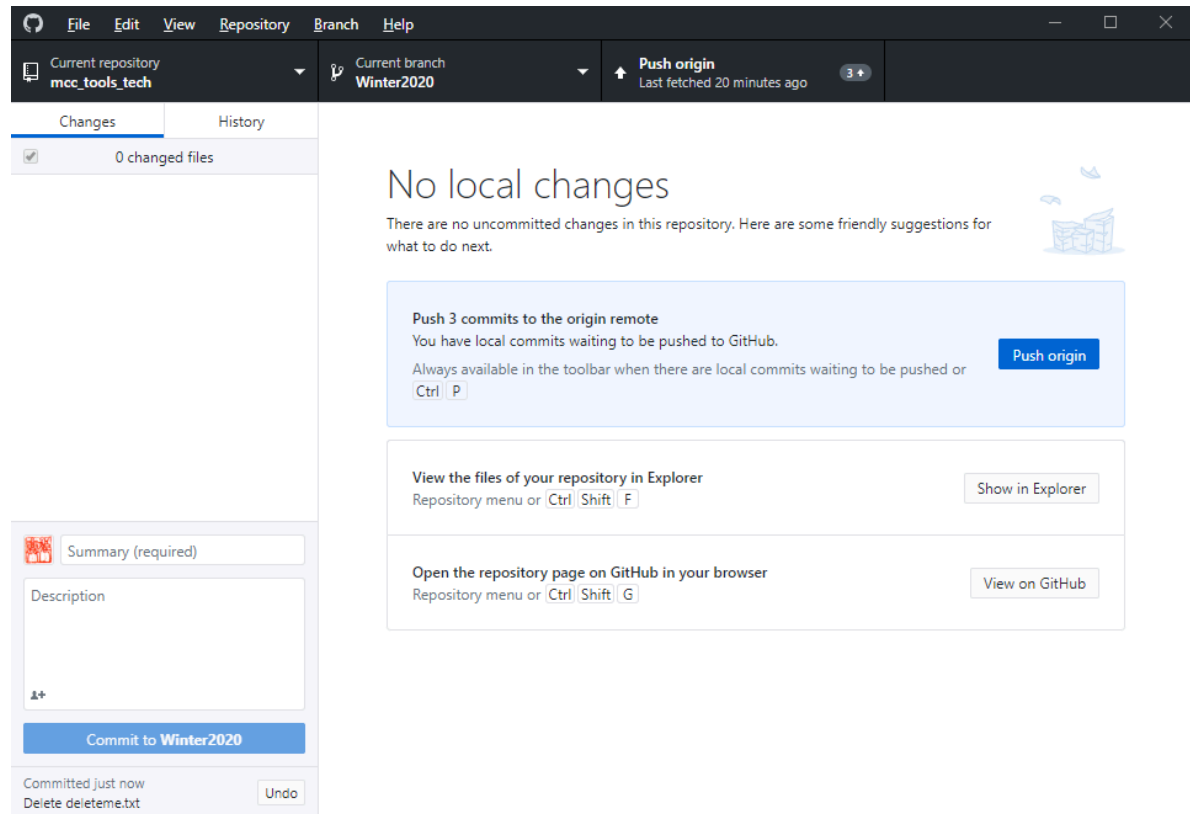
1. Make some change to your local copy of the repository.

In this example, created a new file called `text.txt` with the content `Look a test file`.

When you go to GitHub Desktop after making a change to your local files, you will see the changed files listed in the Changes panel. As you select the files in the Changes panel, their contents display in the right panel.

2. Provide a summary of the change. You can also add more details in the larger text box.
3. Click **Commit to Winter2025**.

Your changes are committed to the local repository.



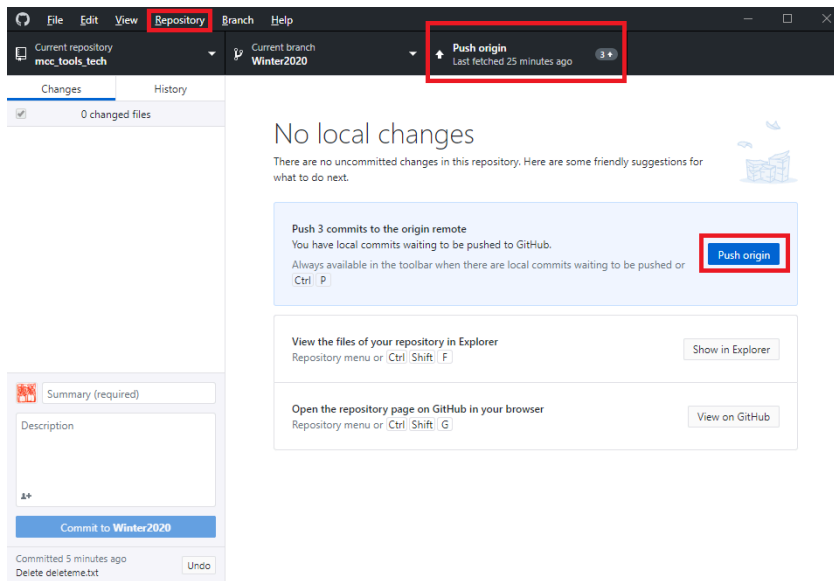
You can now [Push changes to remote repository](#) on page 18.

Push changes to remote repository

After you've committed changes to your local repository, you need to push them to the remote repository.

You usually need to pull any changes into your local repository before you can push changes to the remote repository. See [Pull changes from remote repository](#) on page 16.

GitHub Desktop provides three different methods to push content.



- Click the blue **Push origin** button.
- Click the black **Push origin** button.
- Select **Repository > Push**.

Whichever method you select, the files are pushed to the remote repository (origin).

You are now ready to make a pull request to get your changes in the remote repository to the original forked repository (ZoeLawson/mcc_tools_tech). See [Make a pull request](#) on page 23.

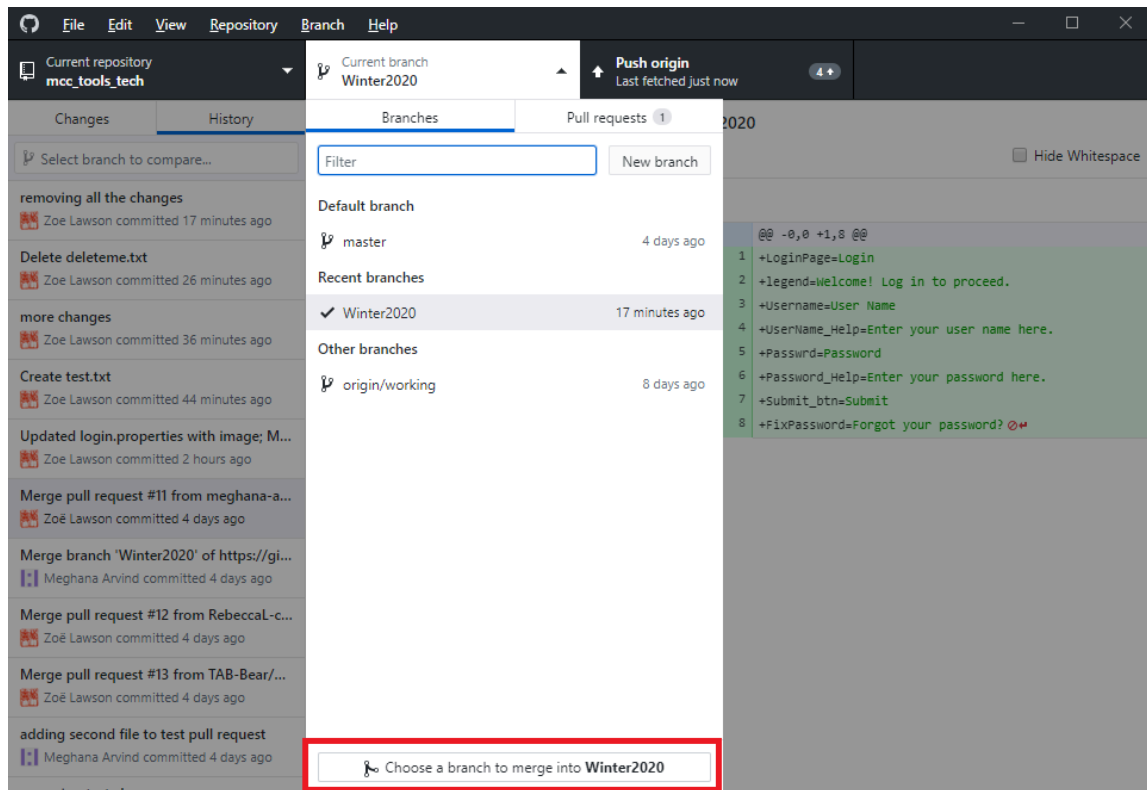
Sync a forked repository

You forked the ZoeLawson/mcc_tools_tech repository. When I make changes to the original ZoeLawson/mcc_tools_tech repository, you need to sync the files to get the changes into your forked version of the repository.

For example, the Week 4 homework is not yet available. When I update the ZoeLawson/mcc_tools_tech repository with the Week 4 homework, you need to sync the repositories to get the changes into yours to do the homework.

Unfortunately, GitHub Desktop does not provide an "easy" sync button. Nor does the user interface use any words like "sync" to help guide you. It follows the Git concept that you are merging changes in from a different branch in a different repository.

1. Select the drop down for the **Current Branch** and select **Choose a branch to merge into Winter2025**.



2. Select **upstream/Winter2025** from the Other branches.
3. Click **Merge upstream/Winter2025 into Winter2025**.

This gets the changes in the "upstream" (the original repository you forked) into your local repository.

You can check your local files to confirm there are some new changes. For example, look for homework from other students or the next week's homework.

4. Click **Push origin** to get the changes into your remote repository.

Start a Pull Request

To make a Pull Request, you have to use GitHub.com. GitHub Desktop provides a shortcut.

There is no way to make a pull request using GitHub Desktop. The only way is to use GitHub.com. GitHub Desktop is kind enough to include a quick way to get to the right page on GitHub.com.

Select the menu **Branch > Create Pull Request**.

The **Comparing changes** page of GitHub.com opens. Follow the instructions in [Make a pull request](#) on page 23, starting with step 4.

GitHub Web Interface

The GitHub web interface is a quick way to look at your repositories in GitHub. You can browse files, make pull requests, and gather other important information from the GitHub web interface.

There are a ton of features in GitHub that this class is not using. You can use the Git Help at <https://help.github.com/en> to learn more.

In this section:

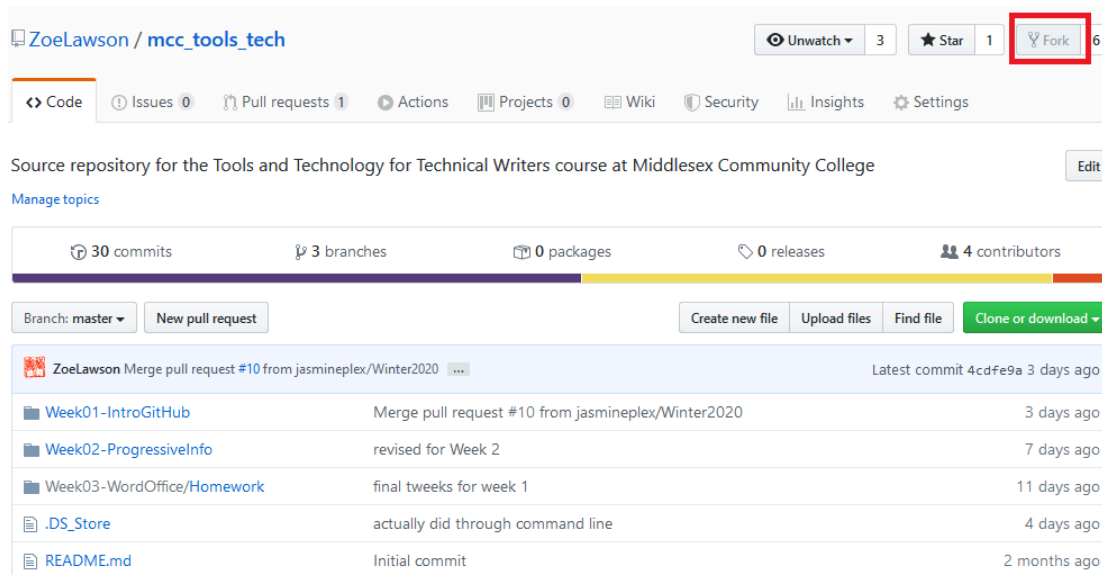
- [Fork a GitHub repository](#) on page 21
This is generally a one time task. (You only need to do it once for this class.) Forking a repository makes a linked copy of the repository in your GitHub account so you can make changes.
- [Make a pull request](#) on page 23
When you have changes in your remote repository you want to get to the original forked repository, you make a pull request. You are asking the owner of the original repository to pull your changes into their repository.
- [Sync a forked repository \(fetch upstream\)](#) on page 24
You can also sync a forked repository using the GitHub web interface using fetch upstream.

Fork a GitHub repository

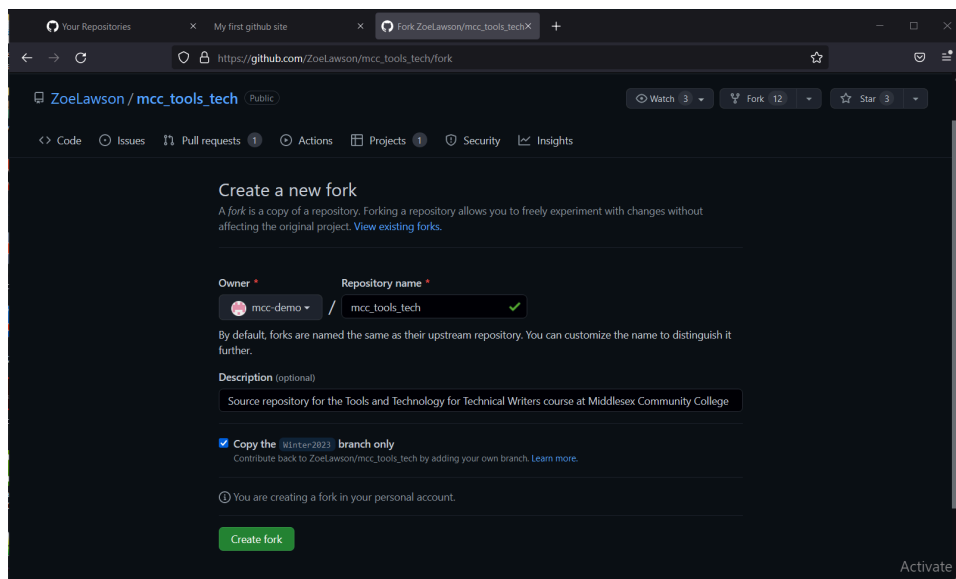
This is generally a one time task. (You only need to do it once for this class.) Forking a repository makes a linked copy of the repository in your GitHub account so you can make changes.

"Fork" is a concept in GitHub. It is not a basic command in Git. In GitHub, you may not know the owner of the content you want to work with. You may not be able to be a contributor to that repository. So you "fork" or make a copy of the repository in your GitHub account. You can then work in the fork, and make a pull request into the original repository.

1. Go to https://github.com/ZoeLawson/mcc_tools_tech and click **Fork**.



You might get a new screen with a few more options to select.



2. Confirm the owner and repository name.

Owner should be your GitHub ID. I recommend keeping the default repository name.

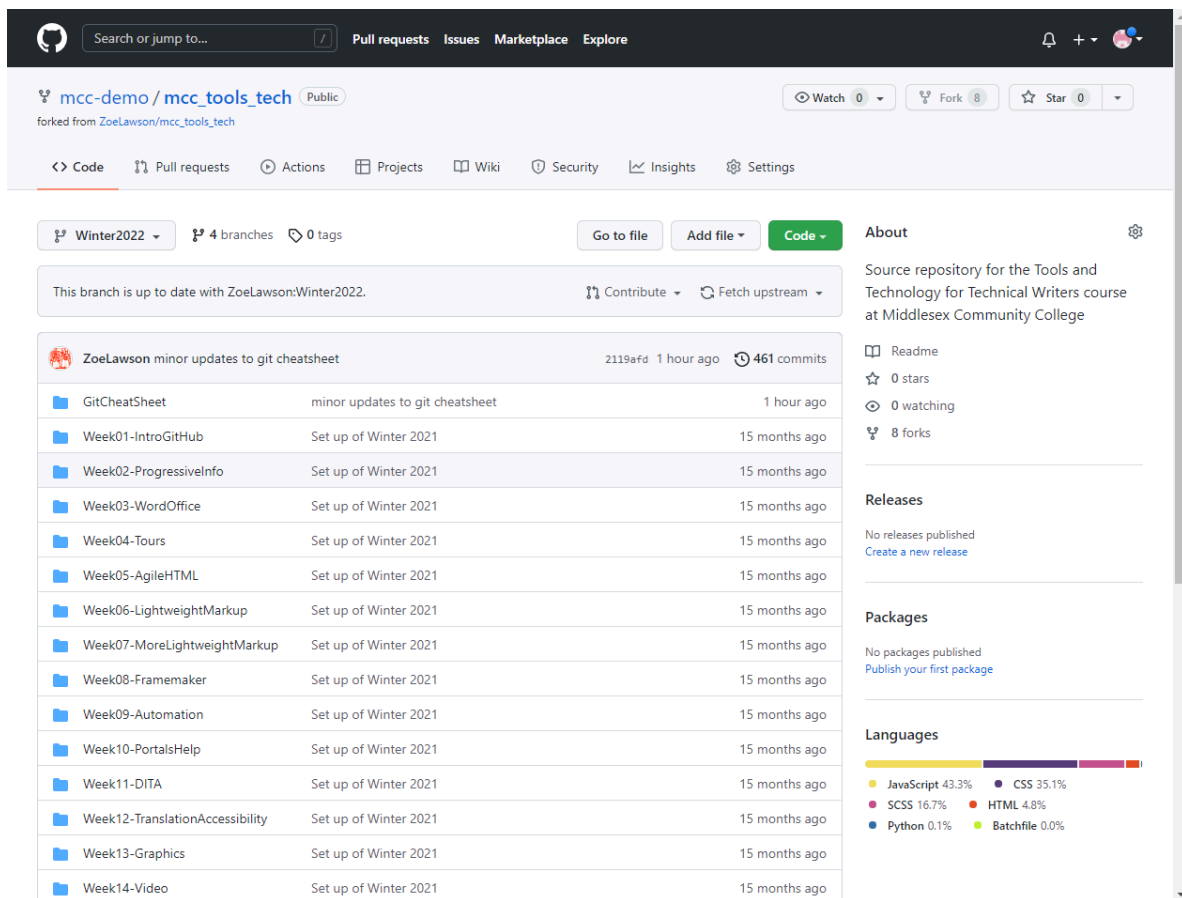
3. If you want, you can revise the description, since your fork is not the "Source repository".

4. Keep the **Copy the Winter2025 branch only** option selected.

If you don't select that, you'll get all the other branches, including all the homework from the previous times I've taught this class. You don't need all those files on your computer.

5. Click **Create fork**.

You have created a copy of the repository in your GitHub account. This copy is currently only available "in the cloud" and isn't on your computer yet.



Things to notice on this screen:

- The name of the repository is *Your_User_Name/mcc_tools_tech*. There is also the label explaining that it was forked from *ZoeLawson/mcc_tools_tech*.
- Notice what branch you're viewing. The default for this year is Winter2025.

After forking the repository, you generally want to clone the repository on your computer. See [Clone Repository](#) on page 14.

Make a pull request

When you have changes in your remote repository you want to get to the original forked repository, you make a pull request. You are asking the owner of the original repository to pull your changes into their repository.

1. Go to the fork of your repository in the GitHub web interface.
2. Make sure you are in the **Winter2025** branch.
3. Click the **New pull request** button.
4. Confirm that the **base** is the original repository (*ZoeLawson/mcc_tools_tech*) and the **compare** is your branch in your fork (*Your GitHub Account Name/mcc_tools_tech*).

5. Add some description of the changes you're requesting to have merged in.

The more descriptive you are, the better for troubleshooting later. In the future, you may not know the person reviewing the pull request. Even if you know the person, they may not be intimately aware of whatever you're working on.

I used to review the pull requests for my team of writers. All of our books are in a single repository. We've had issues with bad push/pull practices in the past where folks have overwritten other people's work. Therefore I try to check if coworker A's pull request only contains files for the books they're currently working on. But, I don't always know everything they're working on. So I ask that people include the book they're working on in the description. Therefore, if they say they're checking in the user guide, but I see changes to files in the installation guide, I can stop and ask questions and try to avoid disaster.

6. Click **Create Pull Request**.

The owner of the original repository gets an email indicating that there's a pull request. You can also see it on the Pull Request tab in the GitHub web interface.. I will then review your request and most likely merge the change. You should receive an email when the request is merged, or if I comment on it.

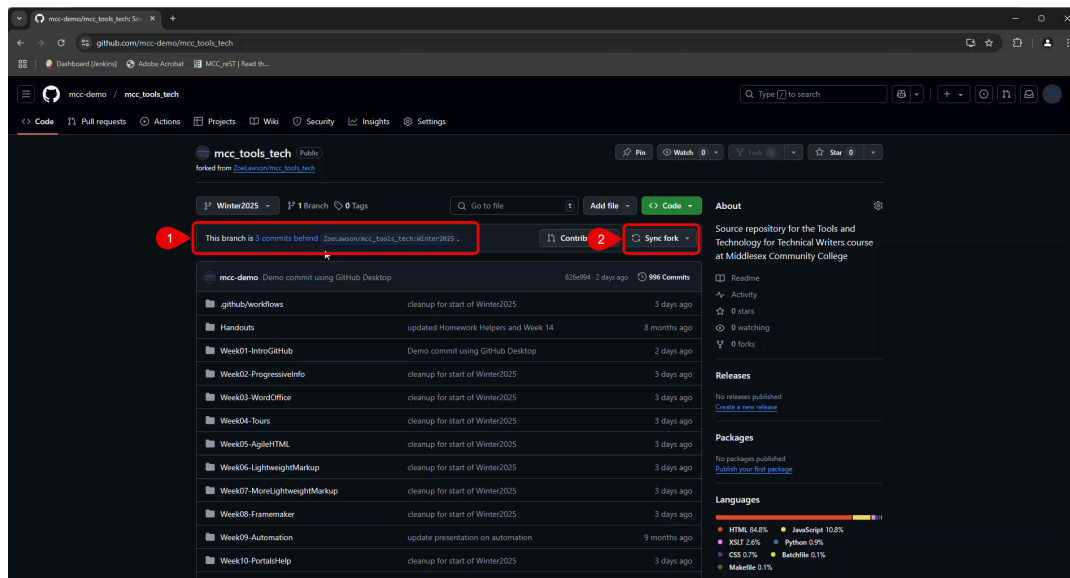
Sync a forked repository (fetch upstream)

You can also sync a forked repository using the GitHub web interface using **fetch upstream**.

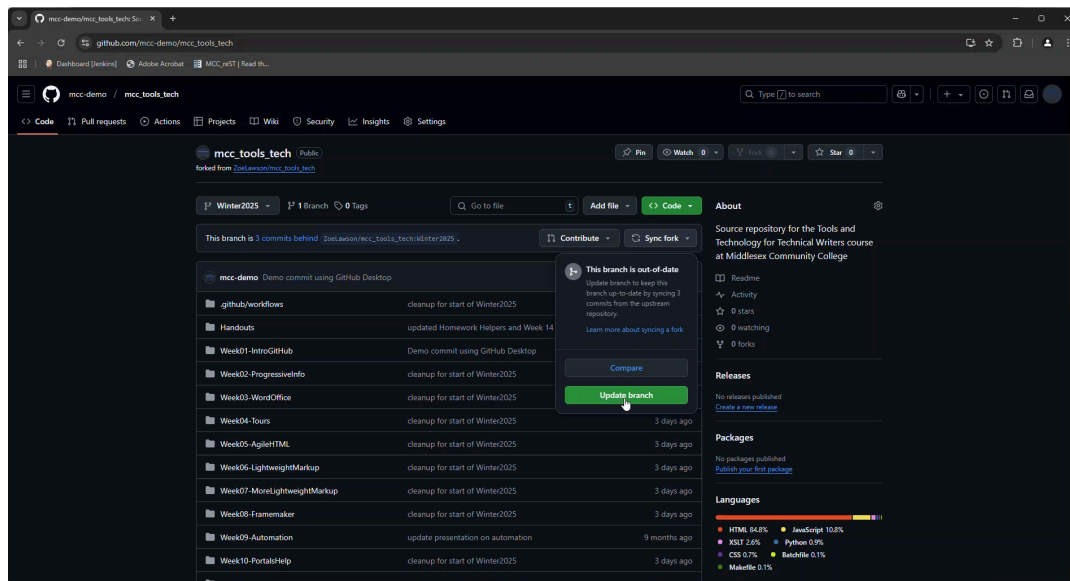
See [Sync a forked repository in GitHub Desktop](#) for more information about what syncing means.

These options may only be available if you can see a message in GitHub along the lines of This branch is *n* commits behind ZoeLawson/Winter2025.

1. From the Code tab of GitHub, click **Sync fork**.

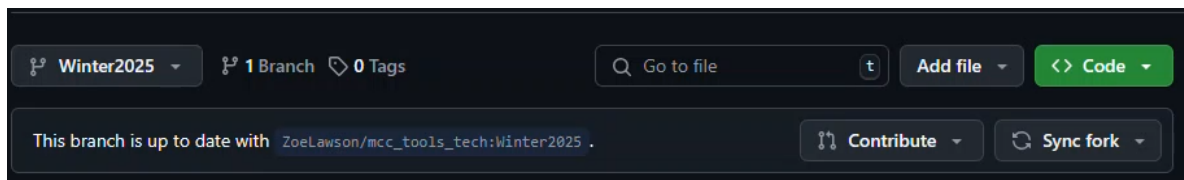


2. Click **Update branch**.



This will change the flow of information from the forked repo (*Your GitHub Account Name/mcc_tools_tech*) to the original repo (*ZoeLawson/mcc_tools_tech*) to the opposite.

These steps will merge the changes from the original repository into your remote repository. You will then need to [Pull changes from remote repository](#) on page 16 to get the files to your local system.



Command Line

A short reference of common (and uncommon) Git commands.

In this section:

- [Common Git Commands](#) on page 26
Here are some of the basic commands you use with Git daily.
- [Uncommon Git Commands](#) on page 28
Here are a few more git commands you use occasionally.
- [Git Status Results](#) on page 30
git status is a command that helps you understand what's happening in your local Git repository.

Common Git Commands

Here are some of the basic commands you use with Git daily.

- [add](#) on page 26
- [commit](#) on page 27
- [pull](#) on page 27
- [push](#) on page 27
- [status](#) on page 28

add

```
git add <files or folder>
```

Add or save the changed files in your local git repository.

You can name specific files or folders, or just use `.` to indicate all the files in this directory and all sub-directories.

This command is relative to the directory you are in. If you are in a "Homework" directory, it will only stage the files in the Homework directory (and all its sub-directories).

Note that these files aren't really remembered by Git until you commit them.

If you make changes to a file after you add them, you will need to add the files again.

For example:

Stage all the files you've changed.

```
git add .
```

commit

```
git commit -m "Witty comment here"
```

Saves the staged files to your local repository.

For example:

```
git commit -m "Week 3 homework"
```

```
[Winter2025 1ac2849] in progress  
1 file changed, 116 insertions(+), 3 deletions(-)
```

The first line gives you the name of the branch, the ID number of the commit, and the commit message.

The second line lists technical details about the changes.

pull

```
git pull <remote repo> <branch>
```

Get the changes from the remote repository into the branch.

You can use a URL or a nickname for the *remote repo*. Use a URL to get from the repository you forked (e.g. ZoeLawson/mcc_tools_tech). Use *origin* for your repository.

For example:

Get changes from the repository you forked from into your local Winter2025 branch

```
git pull https://github.com/ZoeLawson/mcc_tools_tech.git Winter2025
```

Get changes from your fork in GitHub into your local Winter2025 branch

```
git pull origin Winter2025
```

push

```
git push <remote repo> <branch>
```

Get changes from your local repository to the remote repository.

For example:

```
git push origin Winter2025
```

```
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 3.33 KiB | 1.67 MiB/s, done.
Total 15 (delta 10), reused 0 (delta 0)
remote: Resolving deltas: 100% (10/10), completed with 3 local objects.
To https://github.com/ZoeLawson/mcc_tools_tech.git
  6259a47..1ac2849  Winter2025 -> Winter2025
```

The results are a lot of technical details about the data being moved from your local repository to the remote repository.

status

```
git status
```

Get the current status of your local repository.

For example:

```
git status
```

There are a lot of different results from the `git status` command. See [Git Status Results](#) on page 30.

Uncommon Git Commands

Here are a few more git commands you use occasionally.

- [checkout](#) on page 28
- [clone](#) on page 29
- [fetch](#) on page 29

checkout

```
git checkout <Branch Name>
```

Switch between branches. By default, Git always starts with the master branch. In general, you don't want to work in the master branch. In this class, we will constantly be working in Winter2025, so you only need to change branches once.

```
git checkout Winter2025
```

This changes your local branch to the Winter2025 branch.

clone

```
git clone <remote repository URL>
```

Make a copy of a Git repository on your local system in such a way the two repositories know about each other.

```
git clone https://github.com/ZoeLawson/mcc_tools_tech.git
```

This command clones the ZoeLawson/mcc_tools_tech repository to your local system. You should use the fork you created. The URL would include your GitHub user name instead of "ZoeLawson".

fetch

```
git fetch
```

When you clone a repository, you get all the current information about that repository. If people add more branches to the remote repository, your local repository doesn't know about it, until you run a `git fetch`.

This is similar to the `git pull` command. However, the pull can only get branches the local repository knows about.

```
git fetch
```

```
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Total 35 (delta 14), reused 14 (delta 14), pack-reused 21
Unpacking objects: 100% (35/35), done.
From https://github.com/ZoeLawson/mcc_tools_tech
  cdff0ee..43f1608  Winter2025 -> origin/Winter2025
  8fb1229..4cdf9a   master    -> origin/master
```

stash

`git stash`

If you have work that you don't want to commit, but you need to switch to a different branch or restart from scratch, you can stash the changes. The changes are remembered by Git, but not visible to you until you get them back.

I have rarely used this feature on purpose, but have had to help folks recover from using it by accident. I have to look up how to use it every time: <https://git-scm.com/docs/git-stash>

Git Status Results

`git status` is a command that helps you understand what's happening in your local Git repository.

The following are possible results from running the following command:

```
git status
```

There are no changes locally:

```
On branch working
nothing to commit, working tree clean
```

The first line gives you the name of the branch you are on. The second line tells you there's nothing to do.

If you have files changed:

```
On branch working
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
       modified:   Week02-ProgressiveInfo/week2_progressiveinfo.html
```

The first line tells you which branch you are on. The Changes not staged for commit lists all the files that have changed, but are not added. The modified files are usually in red text. The text in parenthesis are suggested commands to help you.

If you have files staged (i.e. you've run `git add .`):

```
On branch working
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
       new file:   handouts/usingGit/git_cheatsheet.ditamap
       new file:   handouts/usingGit/git_commands.dita
```

The first line tells you which branch you are on. The Changes to be committed lists the files you have added/staged, but have not officially told Git to remember. The staged files are often green. The text in parenthesis are suggested commands to help you.

Note: When you're doing a lot of work, you may see both unstaged and staged files when you run the `git status` command.

If you've committed files, but haven't pushed them:

```
On branch working
Your branch is ahead of 'origin/master' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

The first line tells you what branch you're on. The next line tells you that you have changes locally that you haven't pushed up to the remote repository (aka origin)

Troubleshooting

An attempt at providing some work arounds and fixes for common hiccups.

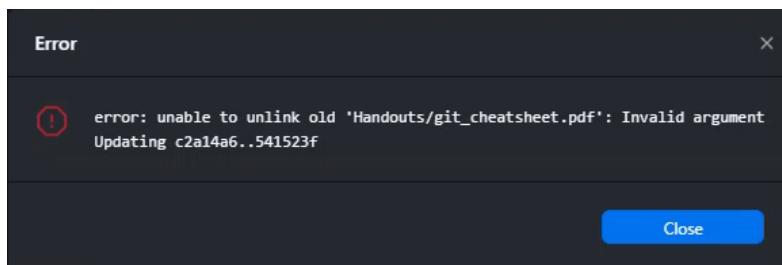
In this section:

- [Unable to unlink file](#) on page 32

Unable to unlink file

Condition

When pulling changes from your fork using GitHub Desktop, you receive the following error:



Cause

GitHub Desktop did not have write access or otherwise has "Access denied" to the file in question. In the above example, the file is `Handouts/git_cheatsheet.pdf`.

A common reason that GitHub Desktop cannot write to a file is if it is open in another application that "locks" the file. The most common example is if a PDF is open in Adobe Acrobat.

Remedy

1. Make sure all files in the repository are not open in any application.

Most commonly, double check that binary files (such as PDFs or images) are not open in Adobe Acrobat or a graphic editor.

Note: Adobe Acrobat is notorious for keeping locks on files, even if the file is closed. Some times I have to close Adobe Acrobat itself to break the lock on the file.

2. In GitHub Desktop, determine the file status.

Unfortunately, this can be quite complicated. When GitHub Desktop was unable to finish pulling the files, it gives up and leaves all your files in a muddle.

For example, if you try to pull again,